

CS 486/686

From Prompting to Fine-Tuning and LoRA

Yuntian Deng

Lecture 22

How to adapt and augment a language model

Search ›

Uncertainty ›

Decisions ›

Learning

Learning goals

- Choose among **prompting**, **RAG**, **tools**, and **fine-tuning**.
- Explain how **RLHF** and **DPO** learn from preferences.
- Derive the shapes and parameter savings of **LoRA**.
- Explain how a large compiler can generate an adapter for a small model.

From L21 to L22

L21 showed how to run one fixed Qwen model.

instructions

change the prompt

knowledge

retrieve evidence

actions

call a tool

behavior

fine-tune weights

Diagnose what is missing, then choose the smallest useful lever.

Suppose we are building a CS486 course helper

STUDENT When and where is the final exam?

HELPER Saturday, August 8, 7:30–10:00 PM in PAC 5. *Source: course schedule.*

The answer must be current, correct, and grounded—not merely fluent.

How should the helper answer?

- 1 Find the relevant schedule entry.
- 2 Place that evidence beside the question.
- 3 Generate the answer and cite the source.

This question needs external knowledge at inference time: RAG.

The adaptation ladder



LoRA is a parameter-efficient way to fine-tune—not a separate adaptation goal.

Prompting changes context, not weights

BEFORE

Answer the student.



AFTER

Answer in two sentences. Quote the date and cite the course schedule.

The prompt controls how to answer; it still does not contain the date.

Prompt templates package behavior

ROLE

You are a concise CS486 course assistant.

EVIDENCE

Use only supplied course context; otherwise say "I don't know."

FORMAT

Answer briefly and cite the source.

A reusable template standardizes behavior across questions.

A better prompt cannot invent a missing fact

When and where is the final exam?

no course schedule in context

retrieve the schedule first

ADAPTATION 1

Add current knowledge

Retrieval-augmented generation (RAG)

RAG puts evidence beside the question

QUESTION

When and where is the final exam?

**RETRIEVED
EVIDENCE**

Final exam: Sat Aug 8, 7:30–10:00 PM, PAC 5.

**GROUND
ED
ANSWER**

Saturday, August 8 in PAC 5. *[course schedule]*

RAG changes the context at inference time—not the weights.

Before questions arrive: store document embeddings



Compute once; store the vector with the original text and its source.

When a question arrives: retrieve, then answer



“Nearest” is the embedding geometry from L18.

RAG on real course facts LIVE

Try the Assignment 3 deadline or final-exam question; inspect the two nearest chunks.

"When is Assignment 3 due?"

"When and where is the final exam?"

When is Assignment 3 due?

retrieve

assignments

Assignment 3 is released
July 9 and is due
Tuesday, August 4, 2026
at 11:59 PM.

0.83

schedule

Assignment 1 is due
Thursday, June 25, 2026 at
11:59 PM (extended from
June 18).

0.74

System: use only the context and
cite the source.

Context: Assignment 3 is
released July 9 and is due
Tuesday, August 4, 2026 at 11:59
PM.

Question: When is Assignment 3
due?

One query benefits from two kinds of match

“Is the exam in
PAC 5
?”

keyword match

finds the exact room code “PAC
5”

embedding match

connects “exam” with “final
examination”

reranker

chooses the strongest evidence

Debug RAG in two checkpoints

1

Did retrieval find the right source?

If not, fix documents, embeddings, or ranking.

2

Is every answer claim supported?

If not, fix the prompt or generation.

RAG reduces unsupported answers; it does not guarantee truth.

ADAPTATION 2

Add actions

Some questions require
computation—not more text.

Language models should not guess exact arithmetic

My scores are 82, 74, and 91 with weights 20%, 30%, and 50%. What is my weighted grade?

model alone

may produce plausible but wrong arithmetic

calculator

```
0.2  
(  
82  
)  
*  
0.3  
(  
74  
)  
+  
0.5  
(  
91  
)  
=  
84.1
```

Let the model choose the operation; let a deterministic tool execute it.

A tool call becomes new evidence

1 • propose `calculator("0.2*82 + 0.3*74 + 0.5*91")`

2 • execute The calculator runs outside the language model.

3 • observe 84.1 returns to the model.

4 • answer Your weighted grade is 84.1%.

Tool use, step by step LIVE

The model calls a calculator instead of guessing the arithmetic.

The model proposes an action; the environment (a calculator) returns real evidence; the model uses it.

step

reset

user

What is my weighted grade? Scores: 82 (20%), 74 (30%), 91 (50%).

Agents repeat the same loop



For consequential actions, permissions and human confirmation belong around the loop.

ADAPTATION 3

Change behavior

When the same behavior must hold
across many inputs, change the weights.

A prompt can be brittle across phrasings

“When is A3 due?”

“Deadline for the third
assignment?”

“How long do I have for
Assignment 3?”

Always answer briefly, cite the evidence, and decline if evidence is missing.

Fine-tuning teaches a durable response pattern through examples.

Supervised fine-tuning learns from ideal responses

INSTRUCTION When is Assignment 3 due?

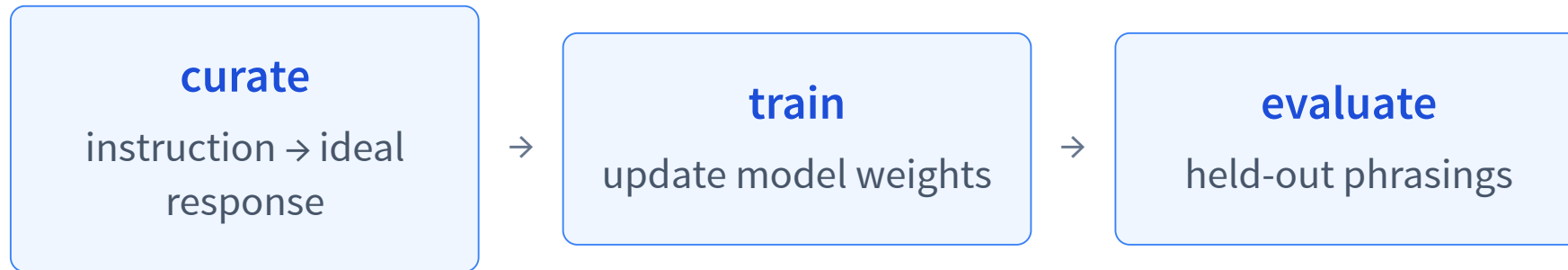
**IDEAL
RESPONSE** Tuesday, August 4 at 11:59 PM. *[course schedule]*

INSTRUCTION Which textbook is required?

**IDEAL
RESPONSE** I do not have supporting evidence in the supplied context.

Training still minimizes next-token loss—on curated behavior examples.

Train on examples; evaluate on new phrasings



warning: success on memorized wording does not imply robust behavior.

Preference data compares two answers

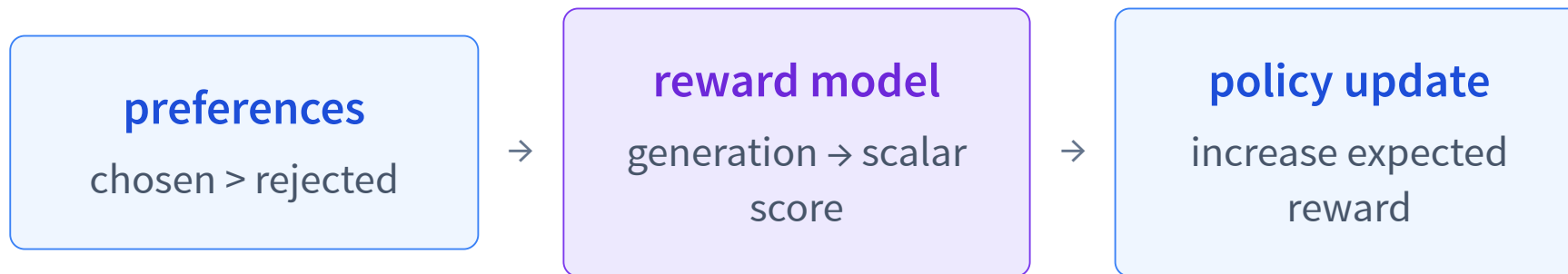
When and where is the final exam?

PREFERRED Aug 8, 7:30–10:00 PM in PAC 5. *[schedule]*

REJECTED It is probably during exam week in August.

Preferences say which answer is better without writing one perfect target token by token.

RLHF learns a separate reward model



Like a learned utility signal from our decision-making unit—but not literally a state-value critic.

DPO optimizes the preference directly

chosen answer

raise its relative log-probability

rejected answer

lower its relative log-probability

direct policy loss

no standalone reward model; no PPO loop

The policy/reference log-ratio acts as an implicit reward score.

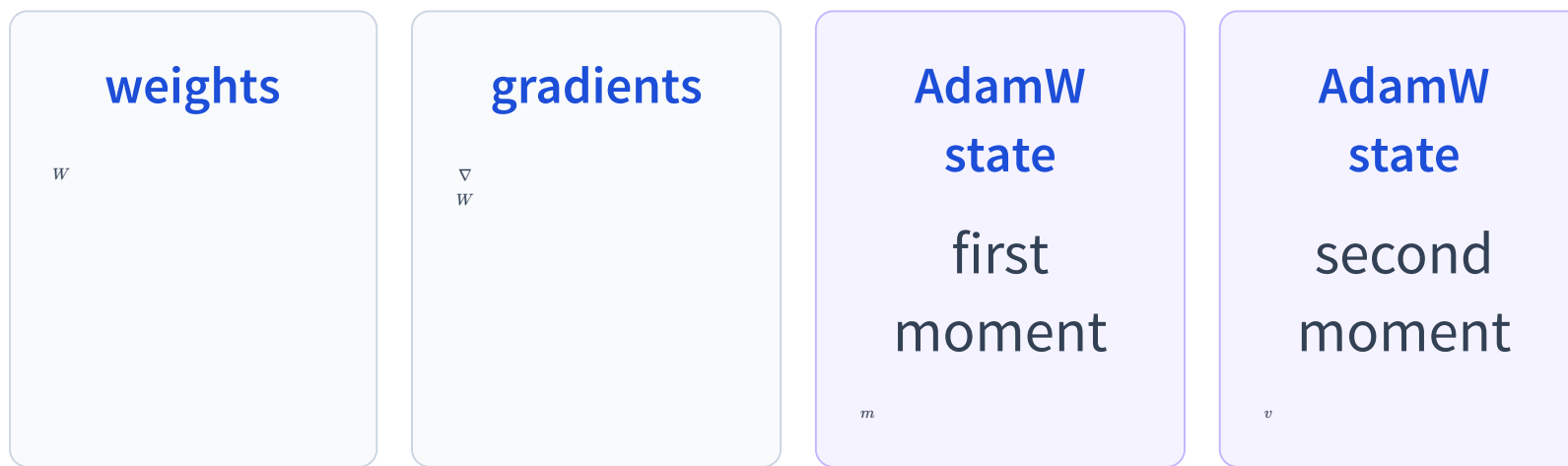
Rafailov et al., *Direct Preference Optimization*, NeurIPS 2023.

PARAMETER-EFFICIENT FINE-TUNING

LoRA: fine-tune fewer parameters

Keep the base weights frozen;
learn a compact update.

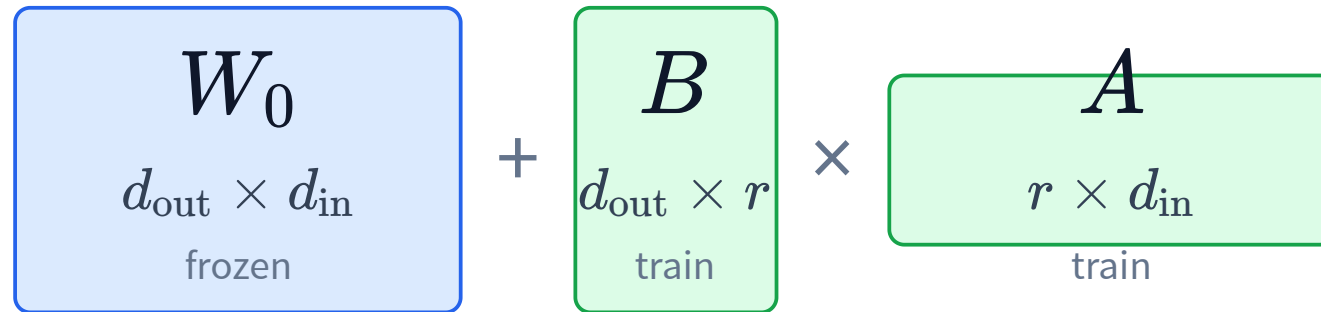
Full fine-tuning stores more than the weights



Roughly four parameter-sized tensors before activations.

Mixed-precision training may also keep an FP32 master-weight copy.

LoRA factorizes a weight update



The diagram illustrates the LoRA weight update factorization. It consists of three main components arranged horizontally: a blue box on the left, a green box in the middle, and another green box on the right. The blue box contains the matrix W_0 with dimensions $d_{\text{out}} \times d_{\text{in}}$ and is labeled "frozen". To its right is a plus sign. The green box contains the matrix B with dimensions $d_{\text{out}} \times r$ and is labeled "train". To its right is a multiplication sign. The final green box contains the matrix A with dimensions $r \times d_{\text{in}}$ and is labeled "train".

$$\Delta W = BA, \quad W' = W_0 + \Delta W$$

d_{in} : input width d_{out} : output width r : adapter rank

For a square $d \times d$ matrix, choose $r \ll d$.

The base stays frozen; only A , B are learned

$$h = W_0 x + \frac{\alpha}{r} B A x$$

full square matrix

trainable parameters

LoRA matrices

$\begin{matrix} d \\ r \\ + \\ r \\ d \\ = \\ 2 \\ d \\ r \end{matrix}$

trainable parameters

Gradients and AdamW states are needed for A , B , not W_0 .

α/r controls the update scale.

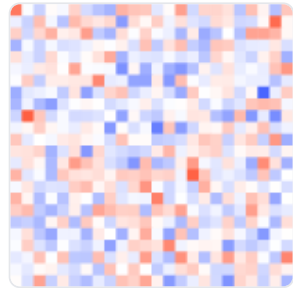
LoRA: a low-rank update LIVE

Raise r from 1 to 24: expressivity grows, but parameter savings eventually disappear.

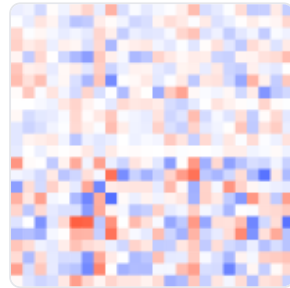
Freeze W_0 ; train $\Delta W = BA$. For this square matrix, LoRA saves parameters only while $r < d/2$.

rank r 4

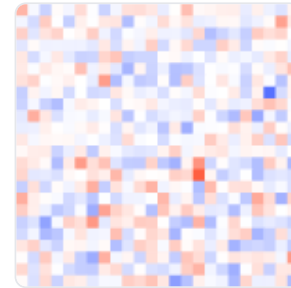
W_0 (frozen)



$\Delta W = BA$



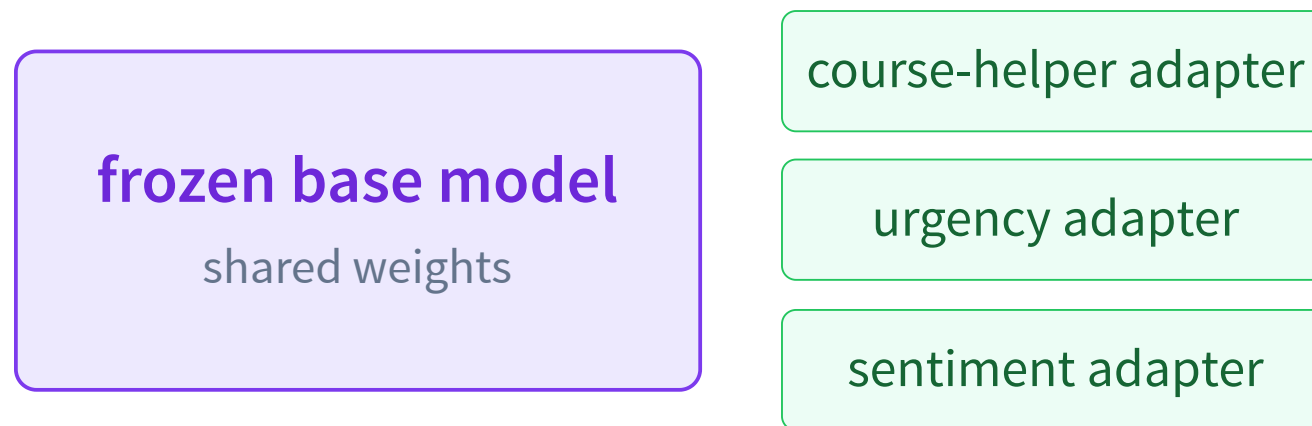
$W' = W_0 + \Delta W$



trainable: **192** vs full **576** (rank $r=4$, $d=24$)

67% fewer trainable parameters

One frozen base can host many adapters



Save and swap small task-specific A , B matrices instead of duplicating the base.

Fine-tune an urgency classifier with LoRA

1 • define

labels: immediate
/wait

2 • collect

message → label
examples

3 • attach

LoRA matrices

4 • train

update only

5 • evaluate

held-out messages;
save adapter

Ordinary LoRA still needs a dataset and a training run for each task.

PROGRAM-AS-WEIGHTS

Automatic adaptation per task

Can a model build the adapter for us?

Ordinary LoRA repeats training for every task

manual adaptation

collect examples

train LoRA

save adapter

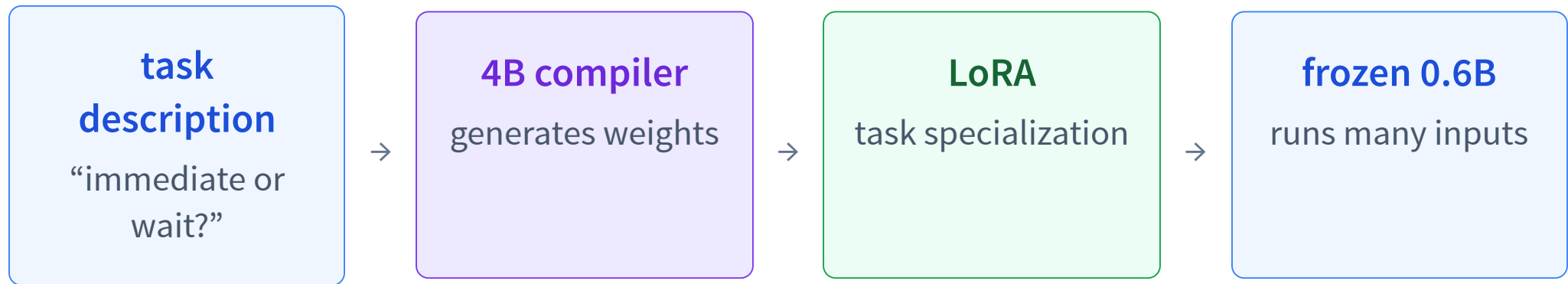
new possibility

describe the task

large model generates LoRA

Use a large model once to build a reusable tool for a small model.

Program-as-Weights compiles a task into an adapter



The expensive model builds the function; the small interpreter executes it repeatedly.

What trains the compiler?

Many tasks, each written as triples (s, x, y) .

task	Classify immediate vs. wait
input	Need your signature today.
target	immediate

task	Classify sentiment
input	The update works beautifully.
target	positive

Across many tasks, the compiler learns how descriptions map to useful weights.

Train end to end through the frozen interpreter



$$\mathcal{L} = -\log P(y \mid x, \text{LoRA}(s))$$

Backpropagate through the frozen interpreter into the compiler.

Deng et al., *Program-as-Weights*, 2026.

Compile an urgency adapter

[LIVE API](#)

Describe `immediate` vs. `wait`, compile remotely once, then infer remotely on new messages.

[1 describe](#)

→

[2 compile](#)

→

[3 run](#)

Describe the function

Classify whether a message needs `immediate` attention or can wait. Return ONLY one of: `immediate`, `wait`.

Input: Thesis defense moved to 3pm; I need your signature today.

Output: `immediate`

Input: Newsletter with events for next month.

Output: `wait`

Input: Production is down for every customer.

Output: `immediate`

[Compile adapter](#)[New task](#)

program ID `not compiled yet`

Run your compiled function

[Example 1](#)[Example 2](#)[Example 3](#)

Please review this whenever you have time next week.

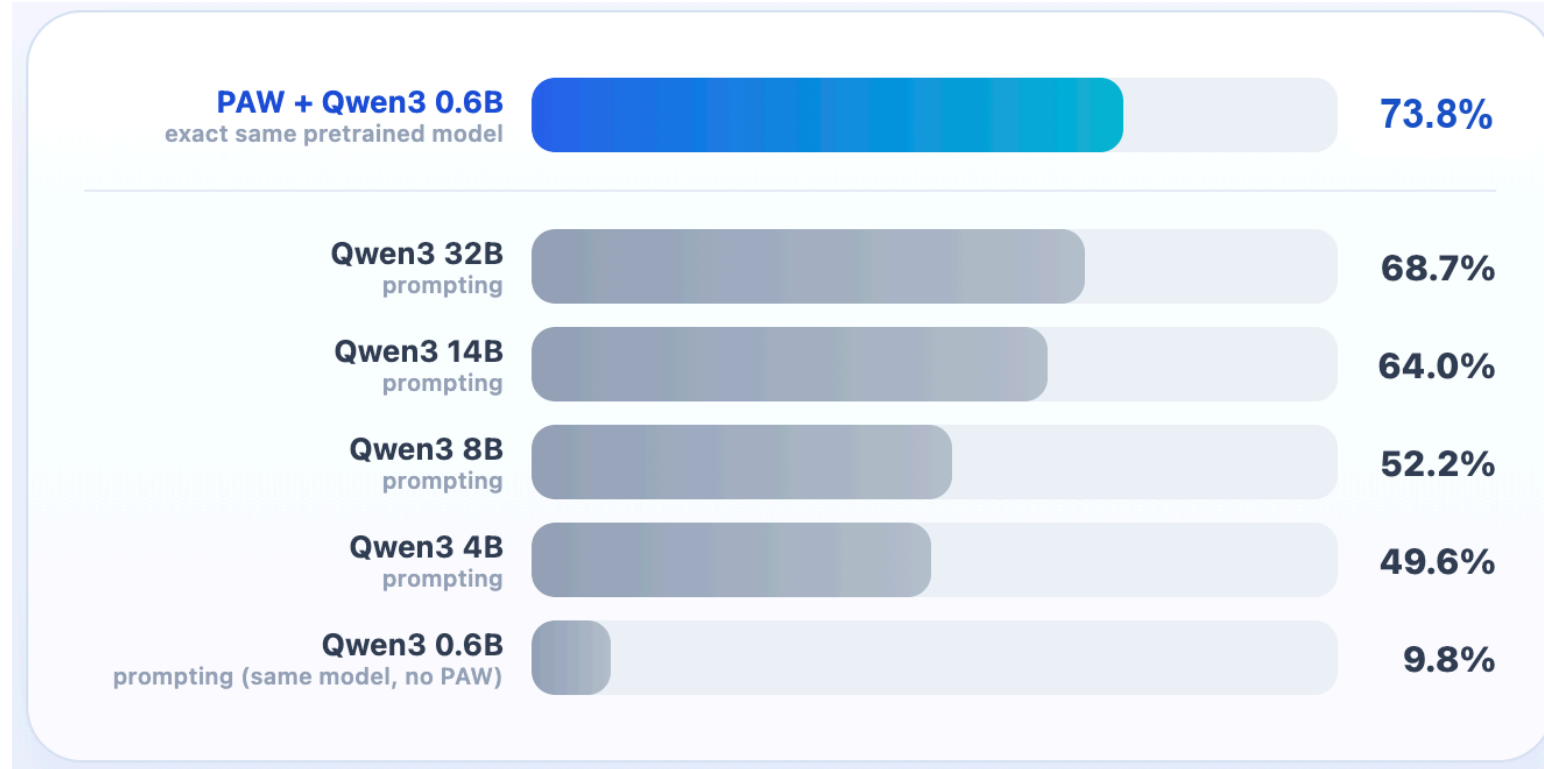
[Run this input](#)

Compile the spec, then run more than one input.

Class demo: hosted compile + hosted inference for zero setup. Do not submit private or sensitive text. The resulting program can also be downloaded and run locally.

Edit the task or examples, then compile an adapter.

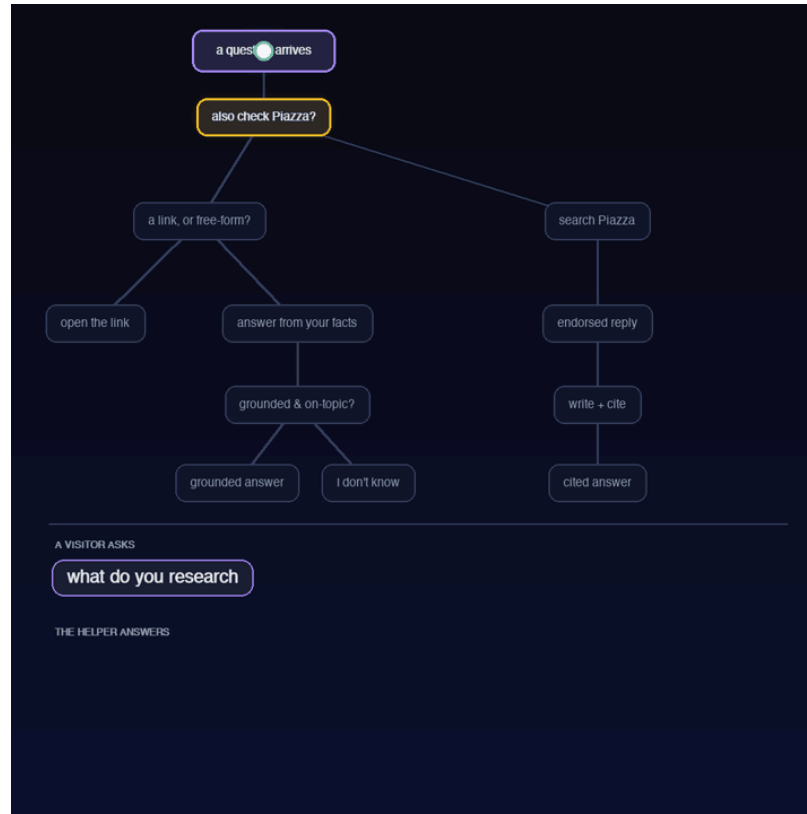
Compiler-generated LoRA can beat a much larger prompt



Verified FuzzyBench exact match: PAW reaches **73.8%** with a frozen 0.6B interpreter, above prompting a 32B model.

Final paper result: 73.78%, rounded to 73.8%.

The course helper is a neural decision tree



Ordinary software controls the branches; PAW neural programs implement semantic branch tests and features.

Compile once; call locally like a Python function

urgency.py

```
import programasweights as paw

# Compile English into a local function.
fn = paw.compile_and_load(
    "Classify if a message needs immediate attention or can wait"
)

# Runs locally – no internet, no API call from here on.
fn("Thesis defense moved to 3pm, need your signature today")
# Returns "immediate"
```

compile once build the adapter **run repeatedly** use the frozen local interpreter

Semantic functions are awkward to hand-code

Count the verbs in: “I can can the can.”

can

modal verb

can

main verb

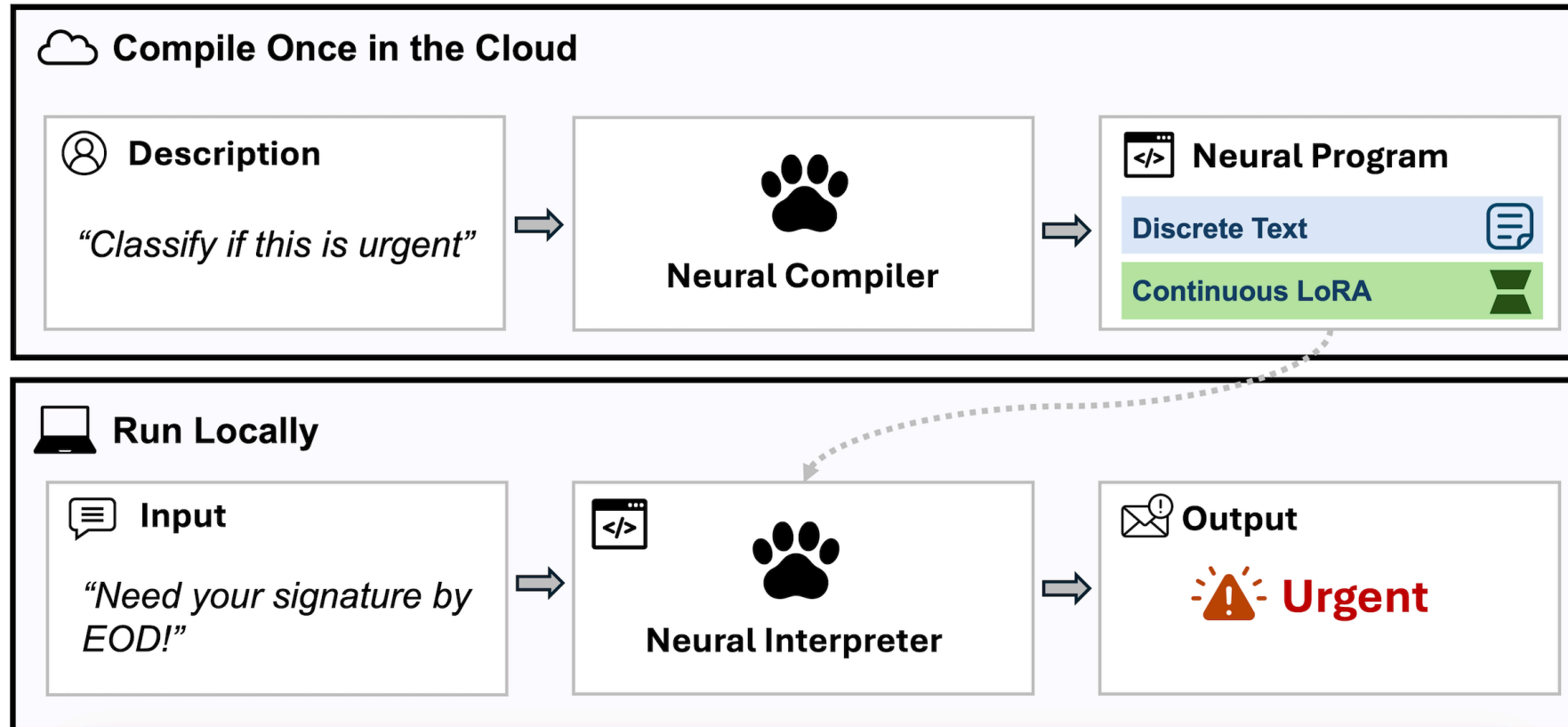
can

noun

answer: 2

Python can express any rule—but learned programs are convenient when meaning, not spelling, determines behavior.

Program-as-Weights: compile once, run repeatedly



The generated LoRA is the reusable task specialization for a shared frozen interpreter.

THE SHIFT

From AI as per-input problem solvers



to AI as per-function tool builders

A large model compiles reusable specialization;
a small model runs it repeatedly.

Next: L23 dissects a vision-language model.